ChatGPT

Defensible Validation of a Custom Browser Performance Profiler Against Chrome DevTools and Lighthouse

Executive framework: what “validation” should mean

The scientifically defensible position is **not** that Chrome DevTools or Lighthouse is ground truth. The stronger position is that they are **recognized reference implementations with partially shared measurement infrastructure**, and that confidence in a new profiler should come from several converging forms of evidence rather than one pairwise statistic.

This distinction matters especially for a CDP/tracing profiler. Chrome DevTools is itself built on Chromium instrumentation, and Lighthouse gathers much of its performance information from Chrome tracing and DevTools protocol data. Lighthouse’s architecture describes a browser driver using Chrome debugging interfaces, gatherers collecting artifacts, `trace.json` supplying most performance characteristics, and a DevTools log supplying important network/page-state information; the resulting trace can itself be inspected in the DevTools Performance panel. [1] Consequently, agreement among a custom CDP profiler, DevTools, and Lighthouse can sometimes mean that all three are interpreting the **same underlying Chromium signals similarly**, not that three independent instruments have confirmed an unknowable true value.

A useful validation model therefore has several distinct layers:

Validation layer	Scientific question	Strongest form of evidence
Construct / measurand validity	Are the tools actually claiming to measure the same quantity?	Explicit operational definitions: event, clock, start point, endpoint, scope, inclusion rules, aggregation
Computational conformance	Given the same raw browser evidence, does the custom implementation compute the expected result?	Replay or analyze the **same trace/ devtools log** through both implementations
End-to-end agreement	Under matched page-load conditions, how different are independently obtained measurements?	Paired repeated runs; paired differences; bias and limits of agreement
Precision / repeatability	How much does each tool vary when nothing intentionally changes?	Replicate measurements under repeatability conditions

Validation layer	Scientific question	Strongest form of evidence
Reproducibility / robustness	Does the relationship persist when legitimate conditions change?	Browser/OS/hardware/configuration strata
Sensitivity to known change	Does the profiler respond correctly when a controlled performance defect is introduced?	Baseline-versus-mutant experiments with known direction or magnitude
Diagnostic validity	Does an issue detector identify defects that are independently known to be present?	Sensitivity, specificity, false-positive rate, effect-detection curves
Decision validity	Are discrepancies small enough for the profiler's intended use?	Predefined practical acceptance tolerances, not merely statistical significance

This layered approach is consistent with metrological terminology. The International Vocabulary of Metrology defines **accuracy** as closeness to the true quantity value and explicitly notes that accuracy is not itself a numerical quantity; it also warns against using "accuracy" and "precision" interchangeably. [2] **Precision** concerns closeness among repeated measurements under specified conditions and is normally quantified through measures such as standard deviation, variance, or coefficient of variation. [3] **Repeatability** is precision under tightly matched conditions; those conditions include the same procedure, system, operating conditions and location over a short time. [4] **Reproducibility** is precision under specified reproducibility conditions in which relevant conditions are deliberately allowed to differ. [5] **Bias** is an estimate of systematic measurement error. [6]

For this project, that yields an important terminological rule:

> **Without an independently justified reference quantity value, use "agreement," "paired difference," "bias relative to the comparator," and "repeatability" rather than claiming absolute accuracy.**

"Error" can still be used when the experiment supplies an independent expected quantity—for example, an exactly generated number of DOM nodes—or when a reference procedure has been independently justified. Otherwise, `custom - Lighthouse` is fundamentally a **method difference**, not automatically an error.

Why correlation cannot validate the profiler

High correlation answers a different question from agreement. Correlation asks whether measurements vary together; agreement asks whether they are sufficiently close to one another in the units that matter. Bland and Altman introduced their method-comparison work specifically because correlation coefficients were being misused for assessing agreement. Their later formulation characterizes agreement through the distribution of paired differences and 95% limits of agreement. [7]

The mathematical problem is simple. Suppose the custom profiler gives $Y = X + 1000$ ms for every page measured by another tool. Pearson correlation can be exactly 1 even though every observation is wrong by

1 second relative to the comparator. Likewise, $Y = 2X$ can produce correlation 1 while exhibiting severe proportional disagreement. Correlation is insensitive to an additive offset and, for positive scaling, does not penalize a multiplicative scale error.

Lin developed the concordance correlation coefficient partly to address this distinction: instead of assessing only linear association, CCC assesses closeness to the 45-degree identity line. [8] CCC is therefore more relevant than Pearson correlation, although even CCC should be supplemental because a single coefficient can conceal magnitude-dependent bias and individual large discrepancies.

Correlation may therefore be reported descriptively, but it should never be the principal validation criterion.

Chrome DevTools and Lighthouse are related, but they are not the same measurement system

Chrome DevTools Performance

The current Chrome DevTools Performance panel is primarily an **interactive diagnostic and trace-analysis environment**. It can record either runtime activity or page-load activity; during load recording, DevTools navigates through `about:blank`, reloads the target, records the load, and automatically stops shortly after load completion. Its trace exposes network activity, main-thread events and event duration/self-duration information. [9]

DevTools also has a live-metrics pathway distinct from an ordinary trace recording. It continuously exposes local LCP and CLS and can record INP once interactions occur; these metrics update as the page is used. [10] DevTools may additionally show CrUX field measurements, but those are aggregated real-user observations and must not be confused with the local trace produced by the test run. [11]

Thus “DevTools measurement” is already ambiguous unless the protocol defines which DevTools surface is being used:

live Web Vitals, **record-and-reload trace**, **runtime trace**, **Network panel**, **Performance insight**, and **CrUX field metric** are not interchangeable datasets.

Lighthouse

Lighthouse is an **automated auditing system**, not primarily an interactive trace viewer. Its architecture separates browser driving, gathering, artifacts, audits, and report generation. The gathered artifacts include trace and DevTools-log data; audits derive higher-level results from those artifacts. [1]

Its Performance score is yet another transformation. Lighthouse maps raw metric measurements into 0–100 scores using log-normal distributions derived from HTTP Archive data, then combines them with metric weights. Current Lighthouse documentation lists FCP, Speed Index, LCP, TBT and CLS as the performance-score components, with weights of 10%, 10%, 25%, 30% and 25%, respectively. [12] Those normalized scores are **not measurements in milliseconds or CLS units**, and the overall Lighthouse Performance score should therefore not be treated as a direct comparator for raw custom-profiler metrics.

Lighthouse also supports different measurement modes. Chrome documents navigation reports for page loads, timespan reports for arbitrary periods—often containing user interaction—and snapshots for a page in a particular state. [13] A comparison protocol must therefore record the Lighthouse mode rather than merely saying “we ran Lighthouse.”

Throttling is a particularly important source of non-equivalence

Lighthouse uses **simulated throttling by default**. Its own throttling documentation says this approach observes an initial page load and models what the load would have looked like under alternate CPU/network conditions; it is fast and deterministic but necessarily imperfect because alternate execution paths must be predicted. [14] Lighthouse also distinguishes its simulation from DevTools-applied throttling and from external/packet-level network control. [14]

Therefore:

> A custom profiler reporting an observed timestamp from an actually executed trace must not be compared uncritically with a Lighthouse result produced under simulated throttling.

That would conflate **measurement-system disagreement** with **different experimental treatments and different computational models**.

A scientifically clean study should normally separate at least two questions:

1. **Observed-measurement comparison:** run Lighthouse under a configuration that avoids post-hoc simulated timing when the custom profiler is being validated against directly observed browser timing.
2. **Lighthouse-compatibility comparison:** if the profiler intends to reproduce Lighthouse itself, separately validate whether it reproduces Lighthouse’s simulation and scoring logic.

Those are different validation targets.

The issue is not theoretical. Lighthouse’s own engineering material describes page loads as draws from a distribution and notes that multiple runs are needed to characterize the experience. [15] General systems-performance research likewise shows that apparently innocuous details of experimental setup can materially bias conclusions. [16]

Shared “Insights” do not make the tools identical

Chrome has increasingly unified the performance-insight logic presented by DevTools and Lighthouse. Chrome 143, for example, shipped Lighthouse 13 in DevTools and described this as completing the work of unifying performance insights across the two tools. [17]

That convergence is useful when validating diagnostic logic, but it does **not** erase differences in execution mode, observation interval, throttling, artifacts, interaction state, or report computation. In fact, when identical diagnostic logic is shared between the tools, agreement between those diagnostics becomes **less independent evidence** than it might superficially appear.

Metric comparability matrix and semantic traps

The categories below use a deliberately strict definition:

Directly comparable means the outputs can represent the same measurand without a conceptual transformation, assuming the same underlying navigation/trace and units.

Conditionally comparable means comparison is legitimate only after specific collection, scope, timing, or computation rules are matched.

Not directly comparable means the quantities are materially different constructs; a numerical comparison would not test agreement.

Almost every end-to-end measurement across two separate page loads is, in practice, at least conditionally comparable because the executions themselves are stochastic.

Metric / output	Custom CDP or trace ↔ DevTools	Custom CDP or trace ↔ Lighthouse	Scientific judgment
FCP timestamp/ duration	**Direct / conditional**	**Conditional**	Same navigation start, frame and paint definition are required; Lighthouse execution and throttling mode must also match.
LCP raw timing	**Direct / conditional**	**Conditional**	Lighthouse explicitly extracts LCP information from Chrome tracing, making same-trace comparison especially valuable. Simulated execution/configuration can still make end-to-end values non-equivalent. [18]
CLS	**Conditional**	**Conditional**	Core algorithm can match, but observation window, lifecycle, interaction exclusions and termination time must match. A DevTools live session can continue observing shifts beyond a Lighthouse navigation run.
INP	**Direct / conditional for a matched interaction sequence**	**Usually not directly comparable to the standard navigation Performance score**	DevTools exposes live INP after user interactions; the Lighthouse performance score is based on FCP, SI, LCP, TBT and CLS rather than INP. [19]

Metric / output	Custom CDP or trace ↔ DevTools	Custom CDP or trace ↔ Lighthouse	Scientific judgment
Long-task duration/count	**Direct / conditional**	**Conditional**	Direct when both operate over the same main-thread trace interval and task definition. Lighthouse TBT is not simply “long-task duration.”
TBT	**Conditional derived comparison**	**Direct / conditional if custom implements exactly the Lighthouse definition**	Lighthouse sums only the portion above 50 ms for long tasks between FCP and TTI. [20] DevTools showing long tasks does not by itself mean it is reporting Lighthouse TBT.
Speed Index	**Usually not directly comparable**	**Only if the custom profiler implements the visual pipeline**	Lighthouse records visual progression and uses Speedline to derive Speed Index. A trace-only profiler has no equivalent metric merely because it has paint timestamps. [21]
Request count	**Conditional**	**Conditional**	Need identical rules for redirects, preflight, frames, cached resources, service workers, failed requests and request types.
Transferred/ network bytes	**Conditional**	**Conditional**	“Transferred bytes,” encoded payload, decoded resource size, body size and total requested-resource size are different quantities. Lighthouse’s enormous-payload audit sums sizes of page resources requested under its audit definition. [22]
Request timing / TTFB	**Direct / conditional**	**Conditional**	Must align navigation/request identity, clock origin, redirects, caching and throttling.
Main-thread scripting/ rendering/ painting time	**Conditional**	**Conditional**	Category mappings and selected intervals must be identical; UI summaries and Lighthouse audit aggregations should not be assumed equivalent merely because category labels look similar.

Metric / output	Custom CDP or trace ↔ DevTools	Custom CDP or trace ↔ Lighthouse	Scientific judgment
CPU task/event durations	**Direct on the same trace**	**Conditional**	Excellent candidate for same-trace computational validation. End-to-end comparison becomes affected by throttling and run variability.
User Timing marks/ measures	**Direct on matching events**	**Direct / conditional**	Lighthouse explicitly extracts User Timing information from Chrome trace data. [23]
DOM-node count / DOM-size diagnostic	**Conditional**	**Conditional or not direct**	Must define whether documents in frames, shadow trees, pseudo-elements or detached nodes are counted. A diagnostic threshold is not equivalent to the underlying count.
Lighthouse metric score	**Not directly comparable to raw custom timing**	Only if custom intentionally reproduces scoring	It is a transformation through Lighthouse's scoring distribution. [24]
Overall Lighthouse Performance score	**Not directly comparable**	Only as a Lighthouse-emulation target	It is a weighted composite, not a primitive browser measurement. [25]
Lighthouse/ DevTools diagnostic or insight flag	**Not a continuous measurement comparison**	**Comparable as a classification/ decision only if rule and input are defined**	Evaluate issue-detection agreement and, preferably, sensitivity/ specificity against independently injected defects.

Identically named metrics can still be different measurements

A metric name is insufficient evidence of measurand equivalence.

For each metric, the validation protocol should create a **measurement contract** containing at least:

Measurement-contract element	Example question
Quantity	Is this timestamp, duration, count, bytes, score or ratio?
Clock / origin	Navigation start? monotonic trace clock? request start?
Start condition	First navigation request? FCP? beginning of recording?

Measurement-contract element	Example question
End condition	Load event? network quiet? TTI? trace stop? user interaction?
Population / scope	Main frame only? all frames? same origin? all resources?
Inclusion rules	Redirects? cached requests? failed requests? service-worker responses?
Aggregation	Sum, maximum, median, session-window maximum, weighted score?
Transformation	Observed value, simulated value, normalized Lighthouse score?
Runtime conditions	CPU/network throttling, viewport, cache, headless/headful?
Software version	Chrome/Chromium, CDP schema, DevTools front-end, Lighthouse package/config?

Several especially dangerous naming collisions deserve explicit tests.

TBT versus "total long-task time." Lighthouse TBT counts only the amount by which qualifying tasks exceed 50 ms and only within the FCP-to-TTI interval. A 90-ms task contributes 40 ms, not 90 ms. [20] Thus a custom field called `totalBlockingTime` that simply sums task durations is not the same metric.

Speed Index versus paint timing. Speed Index is based on visual progression across captured page-load frames and Speedline; no simple timestamp from CDP is automatically equivalent. [21]

LCP raw value versus LCP score. Lighthouse derives LCP information from tracing, but subsequently maps performance metrics through its scoring distribution. [26] A reported "LCP score" and an LCP duration are different variables.

Observed timing versus simulated timing. Lighthouse's default simulated throttling models an alternative execution from initially observed data. [14] An observed custom trace metric and a simulated Lighthouse metric therefore do not constitute a clean method-comparison pair.

CLS under different observation windows. DevTools live metrics update as the page is loaded and subsequently interacted with. [9] A shorter automated navigation capture can therefore legitimately terminate with a different accumulated layout-stability result even if both implementations correctly process every shift they observe.

INP versus TBT. INP reflects actual interaction responsiveness; TBT is a lab-derived blocking proxy based on long tasks in a defined loading interval. Chrome DevTools exposes live INP, whereas Lighthouse's standard Performance score uses TBT rather than INP. [19] They should never be substituted for one another in an agreement analysis.

Historical metrics versus current metrics. Tool versions matter because metric sets evolve. TTI was removed as a Lighthouse metric in Lighthouse 10, and FMP has been removed as of Lighthouse 13. [27] A paper reporting "Lighthouse performance metrics" without its Lighthouse version is therefore incompletely specified.

Statistical comparison when neither tool is perfect ground truth

No single candidate method below should be adopted by convention. Each answers a different question.

Core distinctions

Concept	Meaning for this study	Typical quantitative representation
Accuracy	Closeness to an independently justified true/reference value	Absolute/relative error only when such a value exists
Precision	Scatter among repeated measurements under stated conditions	SD, variance, CV, robust dispersion
Repeatability	Precision with conditions intentionally held nearly constant	Within-tool SD/CV; repeatability coefficient
Reproducibility	Precision after defined conditions are changed	Between-environment variance or hierarchical variance components
Agreement	How close two systems' measurements are to each other	Paired differences, LoA, CCC, coverage within tolerance
Correlation	Strength of linear co-variation	Pearson/Spearman r; not an agreement criterion
Bias	Persistent systematic difference	Mean/median paired difference; regression intercept/slope patterns

The VIM explicitly separates accuracy from precision and defines repeatability and reproducibility as forms of precision under different conditions. [28] Bias corresponds to systematic rather than random error. [6]

Candidate discrepancy and agreement methods

Method	What it answers	Appropriate here?	Key assumptions / limitations
Signed paired difference $d_i = C_i - R_i$	Direction and magnitude of disagreement	**Yes; fundamental**	Requires same measurand and units. Sign depends on arbitrary tool ordering.
Absolute paired difference $\lvert d_i \rvert$	Magnitude regardless of direction	**Yes**	Hides whether disagreement is systematically positive or negative.
Mean absolute difference / MAE against comparator	Typical absolute discrepancy	**Yes, descriptively**	Calling it "error" implies reference status; better call it mean absolute paired difference when neither is truth.

Method	What it answers	Appropriate here?	Key assumptions / limitations
Relative difference $(C - R)/R$	Difference relative to scale	**Conditional**	Asymmetric because R is privileged; unstable near zero.
Percentage deviation	Human-readable scale-normalized discrepancy	**Conditional**	Same denominator issue; can explode for near-zero FCP/TBT/ CLS-like observations.
Symmetric percentage difference $2(C - R)/(C + R)$	Scale-normalized disagreement without selecting a denominator	**Useful option**	Still problematic when both values approach zero and interpretation is less familiar.
Log ratio $\log(C/R)$	Multiplicative disagreement	**Often strong for positive timing/byte metrics**	Requires strictly positive values; translates naturally into ratios/ percent differences.
Mean signed difference / bias	Constant systematic offset	**Yes**	Near-zero average bias can coexist with unacceptable individual disagreement.
Bland–Altman limits of agreement	Range in which most pairwise differences are expected	**Strong candidate**	Classical form assumes reasonably stable variance and an appropriate difference distribution; repeated measurements require extensions.
Regression-based Bland–Altman	Whether bias/ spread depends on magnitude	**Strong when heteroscedasticity exists**	Model form must be justified.
Deming-type errors-in-variables regression	Constant/ proportional relationship when both axes contain measurement error	**Potentially useful**	Requires assumptions about relative error variances; it supplements, not replaces, agreement analysis.
Passing–Bablok regression	Robust/ nonparametric investigation of intercept/slope departures from identity	**Potentially useful**	Assumes an appropriate monotonic/linear relationship; a fitted line still does not quantify individual agreement. The original method was specifically developed for comparing measurements from two analytical methods. [29]

Method	What it answers	Appropriate here?	Key assumptions / limitations
Pearson/ Spearman correlation	Association/ranking	**Supplement only**	Can be perfect despite large constant or proportional bias.
Lin CCC	Concordance around the identity line	**Useful supplementary summary**	One coefficient can hide local or magnitude-dependent disagreement. Lin introduced it as a reproducibility/concordance index tied to the 45° identity line. [8]
ICC	Variance-based reliability under a specified ICC model	**Possible but secondary**	Strongly model- and population-dependent; high between-site heterogeneity can inflate reliability. The exact ICC formulation must be reported.
Paired test of mean difference = 0	Evidence for/against average bias	**Supplement only**	Failure to reject zero bias does not prove equivalence, and says little about individual discrepancies.
Equivalence test such as TOST	Whether a parameter lies within predefined tolerances	**Potentially strong for a defined acceptance question**	Equivalence margin must be justified **before** inspecting results. Schuirmann's TOST work formalized testing within equivalence limits rather than using non-significance as evidence of equivalence. [30]
Coverage within practical tolerance $P(\lvert C - R\rvert \leq \delta)$	Fraction of comparisons sufficiently close for the intended use	**Highly interpretable**	Requires defensible tolerance δ, preferably externally justified and preregistered.

Bland–Altman should be considered, not ritualistically selected

The classic limits are:

$$\bar{d} \pm 1.96 s_d$$

where $d = C - R$, under the usual assumptions for the classical formulation. Bland and Altman describe these as an interval expected to contain roughly 95% of differences. [7]

For the profiler project, this provides information correlation cannot:

- $\bar{d}$ estimates an **additive systematic difference**;
- the spread of differences characterizes disagreement of individual measurements;
- plotting d against $(C + R)/2$ exposes heteroscedasticity or magnitude-dependent bias;
- log-ratio or regression forms can be considered when disagreement increases with measurement magnitude.

However, limits of agreement do **not** themselves say whether agreement is good. A ±300 ms limit could be excellent for one profiler purpose and unacceptable for another. The study therefore needs an externally reasoned **maximum tolerable disagreement** for each metric or use case before any "acceptable agreement" conclusion is made.

Repeated measurements materially change agreement analysis

The ordinary one-pair-per-subject Bland–Altman calculation is not automatically appropriate once each website has several runs. Carstensen, Simpson, and Gurrin explicitly extended agreement models for replicate measurements so repeatability and between-method agreement can be estimated without pretending replicates are independent subjects. [31] Zou likewise developed confidence intervals for limits of agreement when multiple observations exist per individual. [32]

Here, the analogy is:

$$\text{subject} = \text{website} \times \text{controlled condition}$$

and repeated browser loads are **replicates within that subject**, not additional independent websites.

Pooling 30 runs from one site and treating them as 30 independent sites would constitute pseudoreplication and yield confidence intervals that are too optimistic.

A particularly useful hierarchical model

For continuous metric Y, a flexible conceptual model is

$$Y_{s,t,r} = \mu + \alpha_t + u_s + v_{s,t} + \epsilon_{s,t,r}$$

where:

- s indexes website/test case;
- t indexes measurement tool;
- r indexes replicate run;
- α_t represents average tool-level bias;
- u_s captures genuine differences between websites;
- $v_{s,t}$ captures website-specific tool disagreement;
- ϵ captures run-to-run variability.

Such a model can distinguish a tool that is systematically 50 ms higher from one whose discrepancy varies unpredictably from site to site. It can also be extended with browser version, throttling mode, hardware, metric magnitude, and perturbation level as covariates.

The principle is aligned with rigorous systems-benchmarking work emphasizing that nondeterministic systems have multiple sources of variation and that repetitions should be allocated and analyzed according to where uncertainty arises. [33]

Experimental design, repeated runs, and systematic-bias detection

Use a paired blocked design

The central unit should be the **same website under the same defined condition**, measured by both systems. Pairing is valuable because websites differ enormously in inherent performance; comparing unpaired collections would confound website composition with tool effects.

For each test case s, obtain corresponding measurements

$$(C_{s,r}, R_{s,r})$$

under as nearly matched conditions as experimentally possible.

Because two independently launched page loads cannot normally be literally identical executions, a strong design contains **two complementary pairing modes**.

Same-artifact comparison. Save the underlying trace/devtools-log artifact and process that same evidence with the custom implementation and, where technically feasible, the relevant Lighthouse/DevTools computation. Lighthouse's own architecture explicitly exposes gather-mode trace artifacts and states that those traces can be viewed in DevTools. [1] This experiment isolates **parser/computation differences** from browser-run variability.

Independent matched-run comparison. Let each system perform its normal end-to-end measurement on adjacent, controlled executions. This captures the real operational difference experienced by a user of the tools but includes page-load variability, browser state and measurement overhead.

The two experiments answer different questions and should not be collapsed.

Randomize or alternate run order

Running all custom-profiler tests first and all Lighthouse tests afterward invites temporal confounding from thermal state, server/CDN variation, background activity, network changes and page-content changes. Systems research has demonstrated that seemingly harmless experimental details can bias measured performance. [16]

A defensible sequence is randomized or counterbalanced within each website-condition block, for example:

$$C, R, R, C, C, R, \ldots$$

or AB/BA pairs, rather than a fixed `CCCC...RRRR...` design.

For external live websites, measurements should be paired closely in time. For the strongest construct-validation experiments, host immutable/local benchmark pages so that HTML, scripts, resources and backend response behavior are under experimental control.

Repetition should answer three separate questions

Do not select an arbitrary run count merely because “five Lighthouse runs is common.” Lighthouse engineering documentation itself recognizes that page loads are samples from a distribution and that multiple draws are needed. [15] The broader benchmarking literature likewise argues that repetition must be sufficient to estimate uncertainty from nondeterministic execution. [34]

Replicates should support three estimates:

Within-tool repeatability. How much does each tool vary when repeated under the same nominal condition?

For each tool and metric, report appropriate dispersion—SD for reasonably symmetric metrics, CV when scale proportionality is meaningful, and robust alternatives or log-scale estimates for heavily skewed distributions.

Between-tool agreement. On matched website-condition units, what is the paired discrepancy and its uncertainty?

Between-condition reproducibility. Does the conclusion remain when legitimate influence quantities vary —for example browser build, machine, OS, or throttling regime?

The number of replicates should ideally be justified by pilot variance and the desired precision of estimates or confidence intervals, rather than fixed a priori by convention. Rigorous benchmarking methods explicitly frame repetition as a resource to allocate according to sources of uncertainty. [35]

Do not average too early

Suppose a page is run 20 times with each tool. Reducing each tool immediately to one median and then calculating agreement discards information about repeatability.

It is legitimate to study **agreement of medians**, but that is a different estimand:

> “How well do 20-run median measurements agree?”

rather than

> “How well does an individual profiler run agree?”

Both may matter operationally, but they should be labeled separately.

A robust evidence set could therefore include:

- distribution of individual runs;
- within-tool repeatability;
- agreement of individual paired runs;
- agreement of predefined aggregates such as 5-run or 10-run medians;
- uncertainty as the aggregation depth changes.

Detecting systematic bias

Systematic bias should be investigated in several complementary ways rather than inferred from one number.

Constant bias: estimate mean or robust median of $C - R$ with uncertainty. VIM defines measurement bias as an estimate of systematic measurement error. [36]

Proportional bias: plot paired difference against magnitude and examine whether discrepancies increase with average metric value. A log ratio can be preferable when a 10% difference is more meaningful than a fixed number of milliseconds.

Identity-line regression: use an errors-in-variables approach where appropriate. Ordinary least squares implicitly treats its predictor as measured without error, which is a questionable assumption when both browser tools are noisy. Passing and Bablok's original method was designed specifically around equality of measurements from two methods. [29] A Deming analysis is another candidate when its measurement-error variance assumptions can be justified.

Condition-dependent bias: test tool × condition interactions. A custom tool may agree under unthrottled desktop conditions but disagree under CPU slowdown, or may match small pages but undercount large resource graphs.

Website-dependent bias: inspect the distribution of site-specific differences and site × tool interactions. An acceptable grand mean can conceal a systematic failure on JavaScript-heavy, SPA, iframe-heavy or service-worker-controlled pages.

Version-dependent bias: repeat a subset across browser/Lighthouse releases. Chrome's historical documentation shows that Lighthouse metric sets and scoring logic have changed, including removal of TTI and FMP. [27]

Statistical significance and engineering significance must remain separate. With thousands of page loads, a 2 ms average difference can become statistically significant while being completely unimportant. Conversely, a clinically—or here, technically—important ±500 ms spread can coexist with a non-significant mean bias.

Controlled performance defects as independent validation evidence

Controlled perturbation is one of the strongest additions to this validation strategy because it changes the question from:

> “Does my profiler agree with Lighthouse?”

to:

> “Does my profiler detect a deliberately introduced phenomenon whose existence and direction are known independently of Lighthouse?”

This is analogous to mutation-testing ideas. Performance mutation testing explicitly uses artificially injected performance faults to assess and improve the detection power of performance tests. [37]

It does **not** establish absolute accuracy for every metric, but it substantially reduces epistemic dependence on the reference tool.

Network payload perturbation

Create a baseline page and versions that request controlled additional assets, for example nominal increments of 1 MB, 5 MB and 20 MB.

Then study:

$$\Delta_{\text{custom}} = M_{\text{mutant}} - M_{\text{baseline}}$$

rather than only absolute agreement against Lighthouse.

For a byte-counting profiler, particularly strong questions are:

- Does the reported transferred/resource size increase monotonically?
- Is the observed increment close to the expected byte increment under the explicitly defined byte semantics?
- Does it work with compression disabled and enabled?
- Does cache state alter the result in the theoretically expected way?
- Are redirects and service workers classified correctly?

This is important because Lighthouse’s payload audit reports the total size of resources requested by the page under its own audit semantics. [22] A custom profiler should establish precisely which CDP byte quantity it is measuring before comparing numeric values.

Long-main-thread-task perturbation

Inject a controlled synchronous CPU task, for example approximately 100, 300, 600 and 1,000 ms.

The validation target should distinguish at least:

1. task existence;
2. task duration;
3. long-task classification;
4. blocking contribution.

For Lighthouse-style TBT, a nominal 300-ms long task does not automatically imply a 300-ms TBT increase. Under Lighthouse's definition, only time above 50 ms contributes, and only if the task falls inside the FCP-to-TTI interval. [20] Thus a carefully positioned 300-ms task could be expected to contribute approximately 250 ms before other run effects are considered.

This test can uncover an implementation that correctly detects task duration but incorrectly implements the TBT transform.

Layout-shift perturbation

Construct a page that moves a known visible element after initial rendering without qualifying user input and create several shift magnitudes.

Validation can examine:

- whether the layout-shift event is detected;
- whether the affected element is identified;
- whether shift score ordering is correct;
- whether multiple shifts are grouped correctly;
- whether user-triggered variants are appropriately excluded where the metric specifies exclusion;
- whether the custom profiler and DevTools behave consistently when observation windows are identical.

This is stronger than simply reproducing a Lighthouse CLS number because the experimenter controls the **causal manipulation**.

DOM-size perturbation

Programmatically add exact increments such as +100, +1,000 and +10,000 ordinary elements.

This test is particularly useful for exposing definitional problems. If the custom profiler reports the wrong increment even on a simple single-frame document without shadow DOM, there is a direct counting problem. Once that elementary case works, add iframe, shadow-DOM and detached-node variants to determine which population each implementation counts.

Use dose-response, not merely defect/no-defect

A binary mutation is weaker than a graded perturbation. Prefer levels such as

$$0,\ 100,\ 500,\ 1000,\ 5000$$

for added DOM nodes or

$$0,\ 100,\ 300,\ 600,\ 1000\ \mathrm{ms}$$

for blocking work.

Then ask whether the measured effect is:

- directionally correct;
- monotonic;
- approximately proportional where theory predicts proportionality;
- distinguishable from baseline variability;
- stable across repeats.

This produces evidence of **responsiveness** and **effect calibration**, not merely agreement.

Sensitivity and specificity become legitimate when the injected condition supplies the label

For binary issue detectors, an injected defect can provide a known positive condition and an unmodified control a known negative condition. Standard detection terminology then becomes appropriate:

$$Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

NIST defines sensitivity as the probability that a test detects the condition when it is present and specificity as the probability of a negative result when the condition is absent. [38]

For example, if the profiler has a “large network payload” warning, do **not** define Lighthouse’s warning as truth and then call agreement with Lighthouse “sensitivity.” That would only measure classification agreement with Lighthouse.

Instead:

- generate payloads independently above and below the profiler’s declared threshold;
- record true positive, false positive, true negative and false negative results;
- estimate sensitivity and specificity with uncertainty;
- vary payload magnitude around the threshold to characterize the detector’s transition region.

For continuous issue scores, ROC-style analysis is possible when the injected/no-injected status is independently known. NIST’s ROC framework explicitly uses the TP/TN/FP/FN classification and corresponding sensitivity/specificity quantities. [38]

A useful distinction is therefore:

> **Agreement with Lighthouse validates compatibility; controlled defect detection validates responsiveness to an independently established condition.**

Both are valuable, but they are not the same evidence.

Proposed evidence structure, environmental controls, and threats to validity

A defensible study structure without prematurely fixing the final statistic

A strong thesis or publication could organize the empirical evidence into a sequence such as this.

Measurement-definition study. Produce the measurement contracts for every custom metric and map its Chromium/CDP primitive and algorithm to DevTools/Lighthouse counterparts. Exclude metrics whose constructs cannot be aligned.

Trace-conformance study. Feed the same saved traces/artifacts into the custom analysis and reference computations wherever possible. Test exact event identities and derived metrics. This is especially compelling because Lighthouse itself exposes trace artifacts and acknowledges that most performance characteristics derive from them. [1]

Repeatability study. On immutable benchmark pages, repeatedly measure each condition with each tool independently. Quantify each tool's own variation before discussing cross-tool discrepancy.

Paired end-to-end method-comparison study. Use a representative site set with randomized/counterbalanced runs. Report paired differences, absolute discrepancies, bias patterns, appropriate limits or ratios, and uncertainty.

Perturbation study. Use network, CPU, layout and DOM mutants at graded levels. Evaluate directional correctness, effect magnitude, detection probability and, for categorical findings, sensitivity/specificity.

Reproducibility study. Deliberately change selected environment factors to determine whether agreement conclusions generalize.

Decision study. Only after the intended profiler use is explicit should the evidence be judged against metric-specific engineering tolerances.

This gives a much stronger conclusion than "our tool correlates with Lighthouse at $r = 0.97$."

A defensible final statement could instead take the form:

> Under the prespecified Chrome build, Lighthouse configuration and benchmark conditions, the profiler implements the same operational measurand for metrics X and Y; demonstrates specified same-trace computational equivalence; has characterized repeatability; exhibits bounded end-to-end disagreement with no material magnitude-dependent bias over the

tested range; and responds appropriately to independently injected network, main-thread, layout and DOM perturbations.

That is a validation argument rather than a single validation statistic.

Environment and configuration variables that should be frozen or explicitly modeled

Variable	Why it matters
Exact Chrome/Chromium version and build	Trace schemas, web-performance metric implementation, rendering and scheduler behavior evolve.
Exact Lighthouse package/ bundled version	Audits and metric sets evolve; Chrome 143, for example, moved its DevTools Lighthouse panel to Lighthouse 13. [17]
Lighthouse mode	Navigation, timespan and snapshot answer different questions. [13]
Lighthouse throttling method	Simulated throttling is a model based on the observed page load, while applied throttling changes actual execution. [14]
CPU slowdown parameters	Task duration, TBT, LCP and scheduling can change substantially.
Network parameters/ method	Latency, bandwidth, request sequencing and resulting paint timing change.
Headless/headful mode	Rendering and process behavior may differ; it must at minimum be held constant within a primary comparison.
Viewport, device scale factor and form factor	LCP candidate selection, layout and CLS depend on visual geometry.
Cache/storage state	Network counts/timing and service-worker behavior can change.
Browser profile/extensions	Extensions and persisted storage can perturb execution.
Server/content version	Live pages may change between paired runs.
Hardware and OS	CPU speed, power management, scheduling and graphics pipeline affect lab measurements.
Thermal/power state	Can produce run-order-dependent CPU performance.
Background workload	Competes for CPU, memory, network and I/O.
Trace categories / profiling options	Instrumentation configuration affects available events and may itself introduce overhead.
Page lifecycle and stop condition	Determines which requests, long tasks and shifts are observed.

Chrome's own Performance panel documentation makes clear that local metrics reflect the current local device/network environment, while CrUX is a different field-data source. [39] Lighthouse's documentation similarly states that network and CPU throttling are part of its normal execution configuration. [40]

Primary threats to validity

Construct validity. The largest risk is calling two outputs by the same name when they have different scope or computation. TBT/long-task time, Speed Index/paint time, observed/simulated timing, raw metric/score and local/field Web Vitals are prominent examples. [41]

Common-method dependence. A custom CDP profiler, DevTools and Lighthouse can share Chrome tracing/CDP inputs. [1] Agreement therefore cannot automatically be interpreted as independent triangulation of physical truth.

Instrumentation effects. Profiling itself can perturb the measured system. Any trace categories, sampling options or screenshots enabled for one system but not another should be treated as an experimental factor rather than silently ignored.

Temporal and order effects. CDN state, server load, thermal conditions and browser/cache state can change during a study. Systems-performance research shows that environmental details capable of appearing innocuous can introduce systematic evaluation bias. [16] Randomization/counterbalancing is therefore preferable to fixed tool order.

Website instability. Live advertisements, personalization, A/B tests, third-party scripts and backend responses undermine pairability. Immutable benchmarks are preferable for construct validation; realistic public websites remain useful for external validity.

Pseudoreplication. Repeated page loads are not additional independent websites. Replicate-aware agreement methodology is needed. [42]

Heteroscedasticity. Differences may become wider for slower/heavier pages. A single absolute MAE or constant Bland–Altman limit can then misrepresent different parts of the measurement range; magnitude-dependent or ratio analyses should be considered. Bland and Altman's method-comparison work explicitly provides extensions beyond the simplest constant-difference case. [7]

Percentage-error pathology. Percentage discrepancy becomes unstable as the denominator approaches zero, which is particularly relevant for quantities such as CLS, TBT on lightweight pages, and issue counts.

Population-range effects. Very heterogeneous websites can produce an impressive correlation simply because between-site differences dwarf measurement disagreement. This is precisely why association should not substitute for method agreement. [43]

Threshold dependence. Sensitivity/specificity for warnings depends on the threshold and defect prevalence/test population. It should be reported for explicitly defined test cases rather than generalized universally. [38]

Synthetic-defect realism. Injected defects provide strong causal control but may not capture the complexity of real production regressions. Performance mutation testing is promising precisely as an assessment of detection power, but the literature also treats realistic performance mutation design as a

research problem rather than a solved oracle. [44] A balanced study should therefore combine controlled mutants with naturally occurring pages/regressions.

Version drift. A validation against one Chrome/Lighthouse version is not automatically transferable indefinitely. Lighthouse has removed metrics such as TTI and FMP over its evolution. [27] The validation artifact should pin versions and ideally include a small compatibility-regression suite that is rerun after major upgrades.

Priority papers and authoritative sources to obtain

The literature should be collected in three tiers rather than treating all sources as equivalent.

Highest-priority measurement and agreement literature

Bland, J. M., & Altman, D. G. — *Statistical Methods for Assessing Agreement Between Two Methods of Clinical Measurement* (1986). This is foundational for understanding why correlation is an inappropriate substitute for agreement. It should be cited when justifying a paired-difference approach.

Bland, J. M., & Altman, D. G. — *Measuring Agreement in Method Comparison Studies* (1999). This is the higher-priority practical/statistical source for the present project because it develops the limits-of-agreement framework and discusses complications beyond the simplest case. PubMed summarizes the conventional 95% LoA as mean difference ±1.96 SD. [7]

Carstensen, B., Simpson, J., & Gurrin, L. C. — *Statistical Models for Assessing Agreement in Method Comparison Studies with Replicate Measurements* (2008). Essential because your browser experiments will naturally contain repeated measurements; the paper explicitly extends classical agreement models to replicate data and derives both repeatability and limits of agreement. [31]

Zou, G. Y. — *Confidence Interval Estimation for the Bland–Altman Limits of Agreement with Multiple Observations per Individual* (2013). Important for uncertainty around LoA when each test subject/site has multiple observations. [32]

Lin, L. I. — *A Concordance Correlation Coefficient to Evaluate Reproducibility* (1989). Priority source when considering CCC as a complementary agreement coefficient rather than relying on Pearson correlation. [8]

Passing, H., & Bablok, W. — *A New Biometrical Procedure for Testing the Equality of Measurements from Two Different Analytical Methods* (1983). Relevant if robust method-comparison regression is considered for constant/proportional differences. [29]

Schuirmann, D. J. — *A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability* (1987). The subject domain is different, but the statistical idea is directly relevant: equivalence must be tested against a predefined acceptance region, not inferred from failure to find a significant difference. [30]

Highest-priority benchmarking and performance-experiment literature

Mytkowicz, T., Diwan, A., Hauswirth, M., & Sweeney, P. F. — *Producing Wrong Data Without Doing Anything Obviously Wrong!* (ASPLOS 2009). Particularly relevant to browser profiling because it demonstrates how seemingly innocent experimental factors can cause systematic performance-measurement bias. [16]

Kalibera, T., & Jones, R. E. — *Rigorous Benchmarking in Reasonable Time* (2013). Priority source for reasoning about repeated benchmarking, sources of nondeterminism, experiment design and uncertainty rather than relying on single benchmark runs. [33]

Kalibera, T., & Jones, R. E. — *Quantifying Performance Changes with Effect Size Confidence Intervals*. Useful for reporting performance changes with uncertainty rather than only point estimates and percentages. [45]

Delgado-Pérez et al. — *Performance Mutation Testing* (2021). Directly relevant to the proposed injected-defect program; it explores artificial performance faults as a way to assess the detection capability of performance tests. [37]

Highest-priority technical documentation and source material

Chrome DevTools Performance features reference. Treat this as the authoritative operational description for current DevTools recording modes, live Web Vitals and trace interpretation. [9]

Chrome DevTools Performance overview. Important for distinguishing local LCP/CLS/INP observations from field CrUX measurements. [46]

Lighthouse architecture documentation. Probably the single most important technical document for the thesis because it establishes the relationship among CDP/browser driving, gatherers, artifacts, `trace.json`, DevTools logs and audits. [1]

Lighthouse throttling documentation. Essential for preventing invalid observed-versus-simulated comparisons. It explicitly documents simulated throttling as Lighthouse's default and describes its inherent modeling inaccuracy. [14]

Lighthouse Performance scoring documentation. Required whenever interpreting its Performance score or distinguishing a raw timing from its normalized audit score. [47]

Metric-specific Lighthouse documentation, especially LCP, TBT and Speed Index. These establish three substantially different measurement pipelines: trace-derived LCP, long-task-derived TBT, and video/Speedline-derived Speed Index. [48]

Lighthouse release/change documentation and the Chrome DevTools release corresponding to the tested browser. Version pinning is necessary because metrics and diagnostics change; Chrome's documentation records Lighthouse 13 integration and the unification of performance insights. [17]

JCGM International Vocabulary of Metrology. Use this as the terminology authority for accuracy, precision, repeatability, reproducibility, systematic error and bias. [49]

Recommended evidentiary hierarchy for the eventual validation claim

The strongest eventual case would not be:

> "The profiler is valid because its values correlate strongly with Lighthouse."

Nor should it be:

> "The profiler is accurate because Chrome DevTools gives similar values."

A much more defensible evidence chain is:

$$\boxed{\text{operational-definition equivalence}}$$
$$+ \boxed{\text{same-trace computational conformance}}$$
$$+ \boxed{\text{independently characterized repeatability}}$$
$$+ \boxed{\text{paired end-to-end agreement and bias analysis}}$$
$$+ \boxed{\text{controlled perturbation / effect-detection evidence}}$$
$$+ \boxed{\text{reproducibility across prespecified conditions}}$$
$$+ \boxed{\text{agreement judged against application-specific tolerances}}$$

This structure avoids two common methodological errors simultaneously: it does not promote Lighthouse or DevTools to unquestionable ground truth, and it does not mistake correlation for measurement agreement. It recognizes instead that a browser profiler is a **measurement system**, and that a convincing validation is an accumulated argument about what is being measured, how consistently it is measured, how it agrees with established implementations, where systematic discrepancies occur, and whether it responds correctly when the underlying performance phenomenon is deliberately changed.

---

[1] lighthouse/docs/architecture.md at main
https://github.com/GoogleChrome/lighthouse/blob/main/docs/architecture.md?utm_source=chatgpt.com

[2] [VIM3] 2.13 measurement accuracy
https://jcgm.bipm.org/vim/en/2.13.html?utm_source=chatgpt.com

[3] [28] [49] [VIM3] 2.15 measurement precision
https://jcgm.bipm.org/vim/en/2.15.html?utm_source=chatgpt.com

[4] [VIM3] 2.21 measurement repeatability
https://jcgm.bipm.org/vim/en/2.21.html?utm_source=chatgpt.com

5 jcgm.bipm.org
https://jcgm.bipm.org/vim/en/2.25.html

6 36 jcgm.bipm.org
https://jcgm.bipm.org/vim/en/2.18.html

7 43 Measuring agreement in method comparison studies
https://pubmed.ncbi.nlm.nih.gov/10501650/?utm_source=chatgpt.com

8 A concordance correlation coefficient to evaluate ...
https://pubmed.ncbi.nlm.nih.gov/2720055/?utm_source=chatgpt.com

9 10 19 Performance features reference | Chrome DevTools
https://developer.chrome.com/docs/devtools/performance/reference?utm_source=chatgpt.com

11 Monitor your local and real-user Core Web Vitals performance ...
https://developer.chrome.com/blog/devtools-realtime-cwv?utm_source=chatgpt.com

12 24 25 47 Lighthouse performance scoring - Chrome for Developers
https://developer.chrome.com/docs/lighthouse/performance/performance-scoring?utm_source=chatgpt.com

13 What's New In DevTools (Chrome 103) | Blog
https://developer.chrome.com/blog/new-in-devtools-103?utm_source=chatgpt.com

14 41 lighthouse/docs/throttling.md at main
https://github.com/GoogleChrome/lighthouse/blob/main/docs/throttling.md?utm_source=chatgpt.com

15 lighthouse/docs/lantern.md at main
https://github.com/GoogleChrome/lighthouse/blob/main/docs/lantern.md?utm_source=chatgpt.com

16 Producing wrong data without doing anything obviously ...
https://dl.acm.org/doi/10.1145/1508284.1508275?utm_source=chatgpt.com

17 What's new in DevTools, Chrome 143 | Blog
https://developer.chrome.com/blog/new-in-devtools-143?utm_source=chatgpt.com

18 26 48 Largest Contentful Paint | Lighthouse - Chrome for Developers
https://developer.chrome.com/docs/lighthouse/performance/lighthouse-largest-contentful-paint?utm_source=chatgpt.com

20 Total Blocking Time | Lighthouse - Chrome for Developers
https://developer.chrome.com/docs/lighthouse/performance/lighthouse-total-blocking-time?utm_source=chatgpt.com

21 Speed Index | Lighthouse - Chrome for Developers
https://developer.chrome.com/docs/lighthouse/performance/speed-index?utm_source=chatgpt.com

22 Avoid enormous network payloads | Lighthouse
https://developer.chrome.com/docs/lighthouse/performance/total-byte-weight?utm_source=chatgpt.com

23 User Timing marks and measures | Lighthouse
https://developer.chrome.com/docs/lighthouse/performance/user-timings?utm_source=chatgpt.com

27 Time to Interactive | Lighthouse - Chrome for Developers
https://developer.chrome.com/docs/lighthouse/performance/interactive?utm_source=chatgpt.com

29 A new biometrical procedure for testing the equality of ...
https://pubmed.ncbi.nlm.nih.gov/6655447/?utm_source=chatgpt.com

30 A comparison of the two one-sided tests procedure and ...
https://pubmed.ncbi.nlm.nih.gov/3450848/?utm_source=chatgpt.com

31 42 Statistical models for assessing agreement in method ...
https://pubmed.ncbi.nlm.nih.gov/22462118/?utm_source=chatgpt.com

32 Confidence interval estimation for the Bland-Altman limits ...
https://pubmed.ncbi.nlm.nih.gov/21705434/?utm_source=chatgpt.com

33 34 35 Rigorous benchmarking in reasonable time
https://dl.acm.org/doi/10.1145/2491894.2464160?utm_source=chatgpt.com

37 Performance Mutation Testing
https://idus.us.es/bitstreams/53c58383-05ea-4bb4-ad6e-df3c9c8dc3a1/download?utm_source=chatgpt.com

38 ROC CURVE - Information Technology Laboratory - NIST
https://www.itl.nist.gov/div898/software/dataplot/refman1/auxillar/roccurve.htm?utm_source=chatgpt.com

39 46 Performance panel: Analyze your website's performance
https://developer.chrome.com/docs/devtools/performance/overview?utm_source=chatgpt.com

40 GoogleChrome/lighthouse: Automated auditing ...
https://github.com/googlechrome/lighthouse?utm_source=chatgpt.com

44 Performance mutation testing - Delgado-Pérez - 2021
https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.1728?utm_source=chatgpt.com

45 Quantifying Performance Changes with Effect Size Confidence Intervals
https://arxiv.org/abs/2007.10899?utm_source=chatgpt.com